LECTURE 15

Full-Text Search and Elasticsearch

Backend Engineering from First Principles

Detailed study notes • Inverted indexes • Relevance • Search architecture

Backend Engineering from First Principles — detailed study notes

Topic: Search at scale, inverted indexes, relevance scoring, typo tolerance, PostgreSQL full-text search, and Elasticsearch.

1. The problem: searching as data grows

Imagine an e-commerce application with a few thousand products. A simple SQL query can search product names and descriptions:

```
SELECT *
FROM products
WHERE name ILIKE '%laptop%'
   OR description ILIKE '%laptop%';
```

For a small dataset, this may be acceptable. As the catalog grows to millions of products, a broad substring search can become expensive. The database may need to examine many rows and text fields to find matches.

The requirements also become more sophisticated:

- **Speed:** return results quickly, ideally in milliseconds for interactive search.
- **Relevance:** show the most useful results first. A laptop product should rank above an unrelated laptop bag when that better matches the intent.
- **Typo tolerance:** a misspelled query such as `laptpo` should still be able to return results for `laptop`.
- **Interactive suggestions:** support type-ahead experiences while the user is still typing.

A traditional `LIKE`/`ILIKE` substring query is not, by itself, a complete search engine. It can find matching text, but it does not automatically provide sophisticated relevance ranking or typo correction.

2. The librarian analogy

Think of a relational database performing a broad text scan as a librarian who checks every book one by one:

- Read a book's title or content.
- Check whether the requested term appears.
- Continue through the collection.
- Return the matching books.

This is thorough but can be slow for a very large collection.

There is a second problem: matching is not the same as relevance. A book with the query in its title may be much more useful than a book that mentions the same phrase once near the end. A plain substring match does not automatically understand this distinction.

The key change is to build an index that maps terms to the documents containing them.

3. Inverted index

An **inverted index** maps each term to the documents (and often positions) in which that term occurs.

A normal document-oriented view looks like:

```
Document 1: Introduction to Machine Learning
Document 2: The Machine Age
Document 3: Coffee Machine Manual
Document 4: Deep Learning Fundamentals
```

An inverted-index view looks like:

```
machine  -> Document 1, Document 2, Document 3
learning -> Document 1, Document 4
coffee   -> Document 3
deep     -> Document 4
```

The index may also record where a term occurs, such as token positions or offsets. This can help with phrase matching and other search operations.

Why is it called inverted?

Instead of starting with each document and asking, “Does this document contain the term?”, the search engine starts with the term and looks up the documents associated with it. The direction of lookup is inverted.

Indexing and querying

```
INDEXING (usually when data is added or updated)
Documents
   |
   v
Analyze text / split into terms
   |
   v
Build term -> document postings
   |
   v
Store inverted index

QUERY TIME
User query
   |
   v
Analyze query terms
   |
   v
Look up terms in inverted index
   |
   v
Collect candidate documents
   |
```

```
    v
Score and rank results
```

The index requires work and storage to build and maintain. Search engines trade indexing/storage cost for faster query-time retrieval.

4. Apache Lucene and Elasticsearch

The lecture introduces **Apache Lucene** as a foundational inverted-index-based search technology and explains that Elasticsearch builds on Lucene.

Elasticsearch provides a service and API around search capabilities, including:

- Indexing documents.
- Searching indexed text.
- Relevance scoring and ranking.
- Querying structured fields.
- Features useful for type-ahead and typo-tolerant search experiences.

Elasticsearch is not the only option. Modern relational databases such as PostgreSQL also provide full-text search capabilities. The right choice depends on requirements, infrastructure, scale, and operational complexity.

5. Relevance scoring

Finding documents that contain a query term is only the first step. A search engine also needs to decide which matching documents should appear first.

Elasticsearch uses a relevance-scoring system. The lecture discusses **BM25** as an algorithm used for scoring text relevance.

The main ideas introduced are:

5.1 Term frequency (TF)

How often does a query term occur in a particular document?

A term appearing several times in a document can be a signal that the document is more focused on that topic. However, frequency is not the only factor.

5.2 Document frequency (DF)

How common is the term across the whole collection of documents?

A term that appears in nearly every document may be less useful for distinguishing relevant documents than a rarer term.

5.3 Document length

The scoring model considers document length. A long document naturally has more opportunities to contain a term than a short one, so length matters when evaluating term frequency.

5.4 Field boosting

Different fields can have different importance.

For a product or book search, a typical relevance preference might be:

```
title > description > body/content
```

A query term in the title may be more meaningful than the same term appearing only once in a long description or body.

Field boosting lets the application assign greater importance to selected fields. The exact boosts are configurable according to the use case; they are not universally fixed.

Conceptual ranking example

Query: `machine learning`

Document	Match evidence	Expected relevance
Introduction to Machine Learning	Terms in title and repeated in content	Very high
The Machine Age	`machine` in title, weaker topic match	Lower
A book that mentions machine learning once near the end	Weak incidental match	Lower

The example illustrates why ranking is more useful than returning every match in arbitrary order.

6. Typo tolerance and type-ahead search

A search experience should often handle imperfect input.

Example:

```
Intended query: what is trending today
Typed query:    what is treading today
```

A search engine can use techniques such as fuzzy matching to find terms that are similar to the user's input. This can help recover from spelling mistakes.

Typo tolerance is not the same as automatic certainty. Similarity-based matching can return false positives, so it should be tuned to the product's vocabulary and user experience.

Type-ahead / autocomplete

A type-ahead interface displays suggestions while a user is typing.

A simplified flow:

```
User types characters
        |
        v
Frontend sends search/suggestion request
        |
        v
Backend queries search index
        |
        v
Search engine returns ranked suggestions
        |
        v
Frontend displays suggestions
```

For a production implementation, debounce requests, limit result counts, and consider caching popular suggestions to avoid issuing excessive queries for every keystroke.

7. Elasticsearch query language and fields

The lecture describes Elasticsearch's query DSL as JSON-based.

The demo defines two fields:

Field	Mapping type	Intended behavior
`review`	`text`	Analyzed full-text content
`sentiment`	`keyword`	Exact-value matching/filtering

`text` vs `keyword`

- `text`: intended for full-text search. The field is analyzed into searchable terms.
- `keyword`: intended for exact values, filtering, sorting, and aggregations where the whole value should be treated as one token.

Example:

- Search the words in a customer review using `review` as `text`.

- Filter for exactly `positive` or `negative` using `sentiment` as `keyword`.

Choosing the correct mapping is important because it affects how a field is indexed and queried.

8. PostgreSQL vs Elasticsearch

The lecture compares two approaches:

PostgreSQL substring search

Example:

```
SELECT id, review, sentiment
FROM reviews
WHERE review ILIKE '%laptop%';
```

`ILIKE` performs case-insensitive pattern matching in PostgreSQL. The `%` wildcard allows any characters before or after the search term.

This is easy to understand and can be useful for simple search requirements. However, broad substring matching over large text datasets can be expensive and does not provide the same full-text relevance features by itself.

PostgreSQL full-text search

PostgreSQL has built-in full-text search features. It can tokenize and index text and rank matching documents. This may be sufficient when the application already uses PostgreSQL and its search needs are moderate.

Elasticsearch

Elasticsearch is a dedicated search engine built around Lucene. It is useful when the application needs richer search capabilities, relevance tuning, typo tolerance, or a separate search system that can scale independently.

High-level decision guide

Situation	Reasonable starting point
Small dataset and simple substring matching	SQL `LIKE` / `ILIKE` may be enough
Search within a PostgreSQL-backed application	Evaluate PostgreSQL full-text search
Rich relevance tuning, fuzzy matching, suggestions, or search-heavy workloads	Consider Elasticsearch
Elasticsearch already operates as part of the platform's logging/search infrastructure	Reusing it may be operationally sensible

Do not choose Elasticsearch automatically for every search box. It introduces another system to operate and keep synchronized with the primary database.

9. Elasticsearch and the ELK stack

The lecture mentions Elasticsearch's use in log management and the **ELK stack**:

- **Elasticsearch:** indexes and searches data.
- **Logstash:** collects and processes log data.
- **Kibana:** helps explore and visualize data.

Together, these tools are commonly associated with log ingestion, searching, analysis, and visualization.

The same fast indexed-search capability that helps search application content can also help engineers search large volumes of logs.

10. The lecture's 50,000-review comparison demo

The lecture demonstrates a Next.js application comparing PostgreSQL and Elasticsearch using a dataset of approximately 50,000 review records.

Dataset and schema

The CSV contains two fields:

- `review`
- `sentiment` (positive or negative)

The PostgreSQL table is called `reviews` and includes:

- `id`
- `review`
- `sentiment`

The Elasticsearch index is also called `reviews`, with mappings:

- `review`: `text`
- `sentiment`: `keyword`

Data-loading process

- Read the CSV file in a Node.js script.
- Initialize the PostgreSQL and Elasticsearch clients using environment configuration.

- Validate that each record has the required fields.
- Insert PostgreSQL records in batches of 1,000.
- Create or reset the Elasticsearch index.
- Bulk-insert the review documents into Elasticsearch.

Batching avoids trying to insert the entire dataset in one oversized database operation.

Search API flow

The API:

- Reads the search term from the JSON request body.
- Validates that the term is present.
- Starts timing the PostgreSQL query.
- Runs a case-insensitive `ILIKE` search.
- Starts and times an Elasticsearch search against the `reviews` index.
- Streams each result to the frontend as it becomes available.

Streaming lets the UI display whichever backend finishes first rather than waiting for both results before showing anything.

Results shown in the lecture

The presenter reports example timings from the demo:

- For one `laptop` search, Elasticsearch returned in about 1 second while PostgreSQL took about 3 seconds.
- For another broad search, Elasticsearch returned in about 500 milliseconds while PostgreSQL took several seconds (one run was reported around 7.5 seconds).

These are observations from that particular demo environment and dataset, not universal benchmark results. Actual performance depends on hardware, configuration, query shape, indexing, data distribution, network latency, caching, and result size.

11. Important engineering considerations

11.1 Keep the primary database as the source of truth

In many architectures, PostgreSQL stores the canonical application records and Elasticsearch stores a search-oriented representation. The search index can be rebuilt from the primary data.

11.2 Keep the search index synchronized

When a product or review changes in the database, the corresponding search document must eventually be updated or deleted. Common approaches include:

- Application-level indexing after writes.
- Background jobs or queues.
- Change-data-capture or an outbox pattern.

The lecture's demo populates both systems from a script; it does not detail a production synchronization strategy.

11.3 Understand consistency

If database writes succeed but indexing fails, the search index may temporarily be stale. Design retries, reconciliation, and observability for this situation.

11.4 Paginate and limit results

Search endpoints should normally have sensible result limits and pagination or cursor-based continuation. Avoid returning an unbounded number of documents.

11.5 Protect the search API

Validate query length and parameters, restrict expensive query features where appropriate, and apply rate limits. Do not expose unrestricted search DSL directly to untrusted clients.

11.6 Measure realistic workloads

Benchmark with representative data, query patterns, concurrency, and result sizes. Compare not only average latency but also tail latency, indexing throughput, resource usage, and operational cost.

12. Common misunderstandings

- **An inverted index is not a database table scan.** It is a term-to-document lookup structure built ahead of query time.
- **A match is not necessarily relevant.** Ranking determines the order of matching documents.
- **BM25 is not simply “count the word.”** The lecture introduces term frequency, document frequency, document length, and field importance as relevant ideas.
- `text` **and** `keyword` **are not interchangeable.** They support different indexing and query behavior.
- **Elasticsearch is not the only full-text search option.** PostgreSQL also supports full-text search.
- **A demo timing is not a universal performance guarantee.** Test with your own workload.
- **Search infrastructure is not automatically the source of truth.** Keep canonical records in the primary data store unless the architecture deliberately chooses otherwise.

13. Interview questions and answers

Q1. What is full-text search?

A search capability designed to find and rank documents based on terms in their text, rather than relying only on simple substring matching.

Q2. What is an inverted index?

A data structure mapping terms to the documents (and potentially positions) where those terms occur. It lets a search engine find candidate documents by looking up query terms.

Q3. Why can `ILIKE '%term%'` become slow?

A leading wildcard makes broad substring matching difficult to serve with many ordinary indexes. The database may need to inspect many rows and text values, especially at large scale.

Q4. What is Apache Lucene?

A search library that provides core indexing and search functionality based on structures such as inverted indexes. Elasticsearch builds on Lucene.

Q5. What is Elasticsearch?

A distributed search and analytics engine that exposes APIs for indexing and querying documents, with relevance scoring and other search capabilities.

Q6. What is BM25?

A relevance-scoring algorithm used in search systems. The lecture highlights term frequency, document frequency, document length, and field boosting as concepts involved in ranking.

Q7. What is field boosting?

Giving selected fields greater influence in relevance scoring. For example, a match in a product title can be weighted more strongly than a match in its description.

Q8. What is typo tolerance?

The ability to return useful matches even when the user's query contains spelling mistakes, often through fuzzy or similarity-based matching.

Q9. Difference between `text` and `keyword` in Elasticsearch?

`text` fields are analyzed for full-text search. `keyword` fields are treated as exact values and are useful for filtering, sorting, and aggregations.

Q10. PostgreSQL full-text search vs Elasticsearch?

PostgreSQL full-text search can be a good fit when search requirements are closely tied to a PostgreSQL application. Elasticsearch is a dedicated search system with richer search capabilities and independent scaling, but adds operational and synchronization complexity.

Q11. What is the ELK stack?

Elasticsearch, Logstash, and Kibana: a commonly used stack for collecting, processing, searching, and visualizing logs.

Q12. Why stream results from two search backends in the demo?

So the frontend can display each result set as soon as it becomes available instead of delaying the faster backend until the slower one finishes.

14. Practical search-system checklist

- [ ] Is simple substring matching sufficient, or is full-text search needed?
- [ ] Have you evaluated PostgreSQL full-text search before adding a separate engine?
- [ ] Are fields mapped appropriately as `text` or `keyword`?
- [ ] Are title/name fields boosted when they should matter more?
- [ ] Have typo tolerance and relevance been tested with realistic queries?
- [ ] Is the index updated when canonical records change?
- [ ] Can indexing failures be retried and reconciled?
- [ ] Are result size and query complexity bounded?
- [ ] Are latency and indexing throughput measured under realistic load?
- [ ] Is the primary database clearly distinguished from the search index?

15. Last-minute revision sheet

- `LIKE` / `ILIKE` can be fine for simple small-scale substring search, but broad scans become costly at scale.
- Inverted index: **term** → **documents containing the term**.
- Indexing does work ahead of query time to make search faster.
- Apache Lucene provides core search technology; Elasticsearch builds on Lucene.
- Relevance scoring ranks results instead of returning arbitrary matches.
- BM25 concepts highlighted: term frequency, document frequency, document length.
- Field boosting gives greater importance to selected fields such as `title`.
- Typo tolerance helps with misspelled queries; type-ahead gives suggestions while typing.
- Elasticsearch `text` is analyzed; `keyword` is for exact-value behavior.

- PostgreSQL also supports full-text search.
- ELK = Elasticsearch + Logstash + Kibana.
- Keep the primary database and search index synchronized; measure performance on your own workload.

Notes are grounded in the supplied Lecture 15 transcript. The production synchronization, API protection, and benchmarking checklist sections are practical engineering additions, clearly separated from the lecture's demo claims.